

Sprint Retrospective Report — Sprint 3

Music Playlist Manager

Gruppo 04 | Corso: Software Architecture Design | A.A. 2025/2026

Sprint	Data inizio	Data fine	Durata
Sprint 3	11 giugno 2026	18 giugno 2026	1 settimana

Nota: La retrospettiva si concentra esclusivamente sul **processo** adottato durante lo sprint, non sul prodotto realizzato. Le osservazioni seguono il modello *Starfish Diagram* (Start / More / Continue / Less / Stop). Per ogni voce è riportata l'attività concreta che ne è risultata.

Starfish Diagram

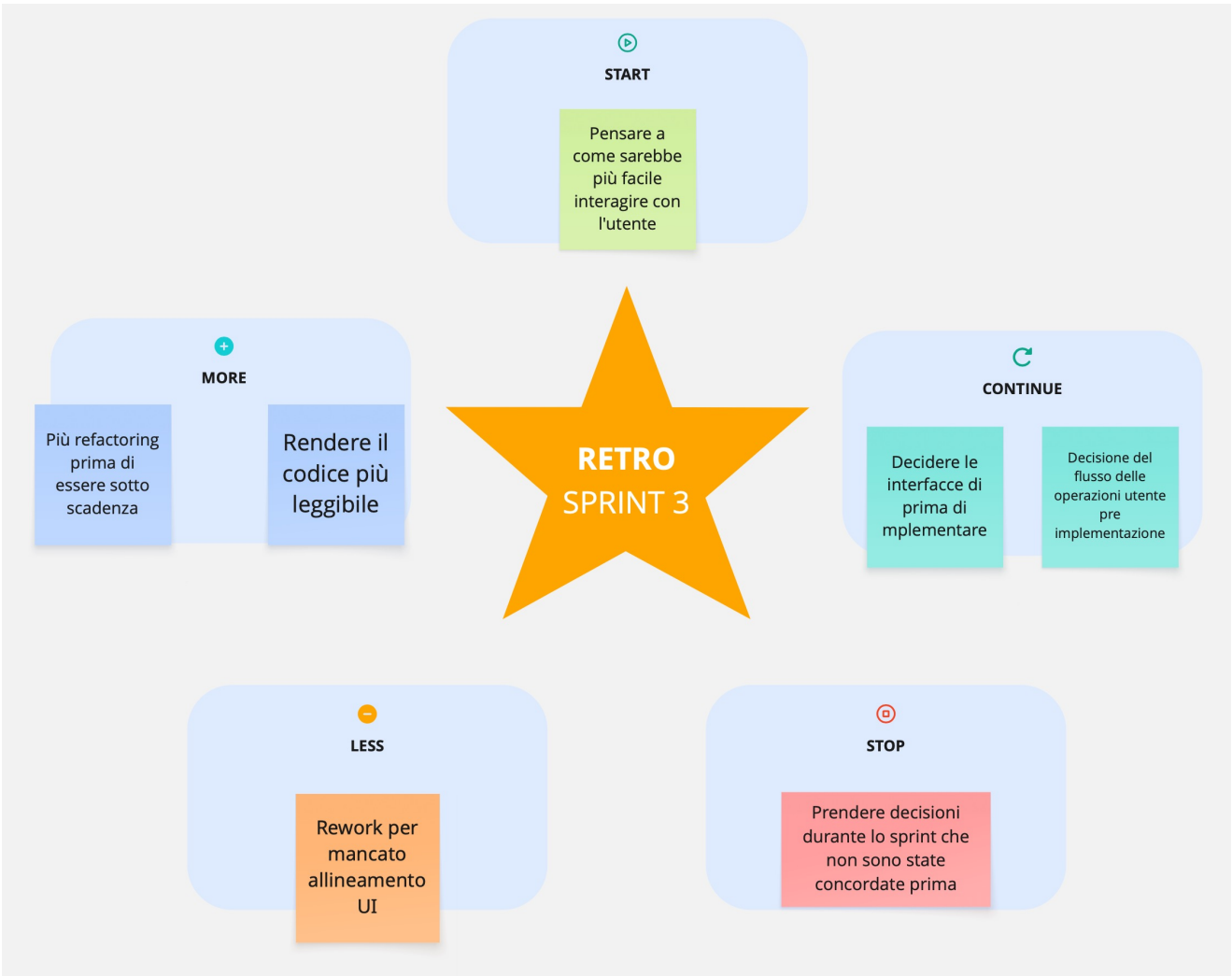


Figura 1: Starfish Diagram — Sprint Retrospective 3

STOP *Cosa dobbiamo smettere di fare*

Prendere decisioni durante lo sprint che non erano state concordate preventivamente con tutto il team.

→ A sprint avviato è emersa la questione se l'operazione moveTrack (US27) dovesse rientrare nel perimetro annullabile gestito dal CommandManager; la decisione, presa in corso d'opera, ha reso necessario un aggiornamento delle interfacce condivise e una redistribuzione parziale dei task tra i membri del team, con conseguente ricalcolo delle stime rimanenti.

LESS *Cosa dobbiamo fare meno*

Rework causato da mancato allineamento nella struttura dell'interfaccia utente tra i diversi membri del team.

→ La vista delle Playlist Automatiche è stata implementata con un layout visivo non coerente con le schede (card) già presenti in PlaylistViewController; il riallineamento grafico è stato eseguito a implementazione conclusa, sottraendo tempo all'integrazione finale e alla scrittura dei test.

CONTINUE *Cosa dobbiamo continuare a fare*

Concordare le interfacce condivise tra i moduli prima di avviare l'implementazione: le interfacce Command, CommandManager e AutoPlaylistGenerator definite collettivamente nel design meeting iniziale sono rimaste stabili per tutta la durata dello sprint, senza modifiche ai contratti già concordati.

Definire il flusso delle operazioni utente tramite mockup o wireframe prima dell'assegnazione dei task: la validazione preventiva della sezione Playlist Automatiche e del pannello di aiuto contestuale (T47) ha evitato rework architetturale e visivo sulle nuove schermate introdotte nello sprint.

MORE *Cosa dobbiamo fare di più*

Pianificare sessioni di refactoring nelle fasi intermedie dello sprint, prima che la pressione della scadenza riduca il tempo disponibile per interventi strutturali.

→ PlaybackController e PlaylistDetailViewController avevano accumulato complessità non risolta dallo Sprint 2; affrontare il debito tecnico nelle giornate intermedie dello sprint, anziché solo a ridosso della consegna finale, avrebbe ridotto il rischio di regressioni in fase di integrazione e semplificato la revisione del codice da parte degli altri membri.

Rendere il codice più leggibile tramite estrazione di metodi ausiliari e commenti Javadoc esplicativi.

→ La logica dello stack undo/redo nel CommandManager e i metodi di costruzione automatica delle playlist nell'AutoPlaylistGenerator hanno raggiunto una lunghezza e una complessità tali da rendere difficoltosa la code review da parte degli altri membri; la suddivisione in metodi privati documentati con Javadoc avrebbe mitigato il problema.

START *Cosa dobbiamo iniziare a fare*

Porsi sistematicamente la domanda "come rendiamo questa funzionalità più semplice e immediata per l'utente?" prima di iniziare l'implementazione, anziché limitarsi a soddisfare i criteri di accettazione in modo tecnico.

→ Funzionalità come la generazione automatica delle playlist (US25) e il pannello di aiuto contestuale (T47) sono state implementate rispettando i requisiti tecnici, ma senza chiedersi in anticipo quale fosse il modo più naturale per l'utente di scoprirle e utilizzarle; adottare questo punto di vista prima di scrivere codice avrebbe portato a scelte di layout e interazione più intuitive, riducendo la necessità di aggiustamenti a posteriori.

Action Items (Sprint 3 → Lezioni Apprese)

Azione

Definire esplicitamente il perimetro dei pattern (es. scope dell'undo nel **CommandManager**) nel design meeting di inizio sprint, prima dell'implementazione.

Pianificare sessioni di refactoring intermedie nello sprint, non solo a ridosso della consegna finale.

Condurre in anticipo una riflessione su come rendere ogni funzionalità semplice e immediata per l'utente, prima di procedere con il design tecnico e l'assegnazione dei task.
